

COMP 3069

Computer Graphics

Lab 9: Light

Autumn 2025



The University of
Nottingham

UNITED KINGDOM · CHINA · MALAYSIA

0

Contents

- ◆ Specify vertex normals correctly
- ◆ Set up VAO and shader variables for lighting
- ◆ Implement ambient, diffuse and specular and combine them in an implementation of the phong lightingmodel.
- ◆ Use the phong lighting model
- ◆ Recognise the effect of attenuation
- ◆ Write a fragment shader for per fragment lighting



The University of
Nottingham

UNITED KINGDOM · CHINA · MALAYSIA

1

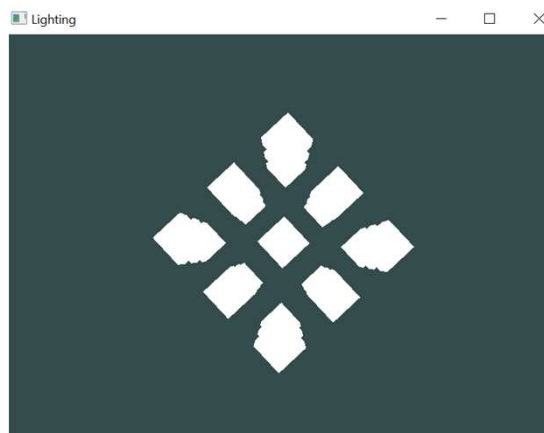
1

Set-up of Lab 9

- ◆ Using the instructions in Lab 1, create Lab9 project
- ◆ Copy `Lab9.cpp` code to the `Lab9.cpp` in the new project
- ◆ Copy the `camera.h`, `ModelViewerCamera.h`, `shader.h`, `util.h`, and `window.h` and add them to the Header Files
- ◆ Copy `phong.vert` and `phong.frag`, and add them to the Source Files

Default Display

- ◆ There are 27 cubes and there is no light or colour yet



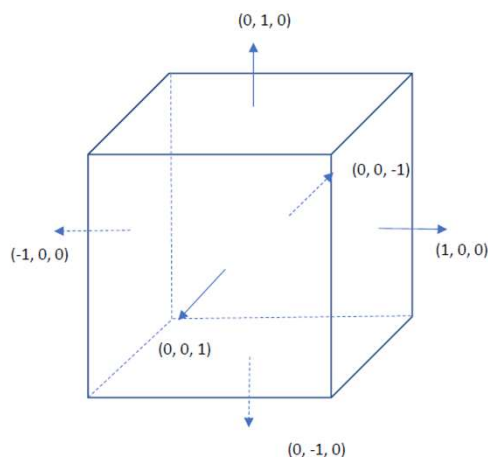
Setup Vertex Normal Attributes

- Each vertex, of each triangle, on each face, needs to have a normal attribute

```
21 float vertices[] =
22 {
23     //back face
24     //pos          //col          //normal
25     -0.5f, -0.5f, -0.5f, 1.0f, 1.0f, 1.0f, 0.0f, 0.0f, -1.0f,
26     0.5f, -0.5f, -0.5f, 1.0f, 1.0f, 1.0f, 0.0f, 0.0f, -1.0f,
27     0.5f, 0.5f, -0.5f, 1.0f, 1.0f, 1.0f, 0.0f, 0.0f, -1.0f,
28     0.5f, 0.5f, -0.5f, 1.0f, 1.0f, 1.0f, 0.0f, 0.0f, -1.0f,
29     -0.5f, 0.5f, -0.5f, 1.0f, 1.0f, 1.0f, 0.0f, 0.0f, -1.0f,
30     -0.5f, -0.5f, -0.5f, 1.0f, 1.0f, 1.0f, 0.0f, 0.0f, -1.0f,
31
32     //front face
33     -0.5f, -0.5f, 0.5f, 1.0f, 1.0f, 1.0f, 0.0f, 0.0f, 1.0f,
34     0.5f, -0.5f, 0.5f, 1.0f, 1.0f, 1.0f, 0.0f, 0.0f, 1.0f,
35     0.5f, 0.5f, 0.5f, 1.0f, 1.0f, 1.0f, 0.0f, 0.0f, 1.0f,
36     0.5f, 0.5f, 0.5f, 1.0f, 1.0f, 1.0f, 0.0f, 0.0f, 1.0f,
37     -0.5f, 0.5f, 0.5f, 1.0f, 1.0f, 1.0f, 0.0f, 0.0f, 1.0f,
38     -0.5f, -0.5f, 0.5f, 1.0f, 1.0f, 1.0f, 0.0f, 0.0f, 1.0f,
39 }
```

Normal Vectors on a Cube

- Each vertex on that face has the same normal vector



VAO Setting

- ◆ Interpret the vertex attributes

```
149 glBindVertexArray(VAO);
150 glBindBuffer(GL_ARRAY_BUFFER, VBO);
151 glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
152 glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 9 * sizeof(float), (void*)0);
153 glEnableVertexAttribArray(0);
154 glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 9 * sizeof(float), (void*)(3 * sizeof(float)));
155 glEnableVertexAttribArray(1);
156 glVertexAttribPointer(2, 3, GL_FLOAT, GL_FALSE, 9 * sizeof(float), (void*)(6 * sizeof(float)));
157 glEnableVertexAttribArray(2);
```

Light Direction

- ◆ Create a variable to store the light direction vector

```
15 //light direction variable here
16
17 glm::vec3 lightDirection = glm::vec3(0.1f, -0.81f, -0.61f);
18 SCamera Camera;
```

Assign Values to Variables

- ◆ Copy the vectors to uniforms in the shaders: i) light direction vector, ii) light colour vector, and iii) camera position vector

```
182 glUniform3f(glGetUniformLocation(shaderProgram, "lightDirection"), lightDirection.x, lightDirection.y, lightDirection.z);
183 glUniform3f(glGetUniformLocation(shaderProgram, "lightColour"), 0.f, 0.f, 1.f);
184 glUniform3f(glGetUniformLocation(shaderProgram, "camPos"), Camera.Position.x, Camera.Position.y, Camera.Position.z);
```

In the vertex shader

```
1 #version 330 core
2
3 layout(location = 0) in vec3 aPos;
4 layout(location = 1) in vec3 aCol;
5 layout(location = 2) in vec3 aNor;
6
7 uniform mat4 model;
8 uniform mat4 view;
9 uniform mat4 projection;
10
11 out vec3 col;
12 out vec3 nor;
13 out vec3 FragPos;
14
15 void main()
16 {
17     gl_Position = projection * view * model * vec4(aPos, 1.f);
18     FragPos = vec3(model * vec4(aPos, 1.f));
19     col = aCol;
20     nor = mat3(transpose(inverse(model))) * aNor;
21 }
```

In the fragment shader

```
1  #version 330 core
2
3  in vec3 col;
4  in vec3 nor;
5  in vec3 FragPos;
6
7  uniform vec3 lightColour;
8  uniform vec3 lightDirection;
9  uniform vec3 camPos;
10
11 out vec4 fragColour;
12
13 void main()
14 {
```

Implementing Directional Light

```
14 float CalculateDirectionalIllumination()
15 {
16     float amb = 0.1f;
17
18     vec3 Nnor = normalize(nor);
19     vec3 Ntolight = normalize(-lightDirection);
20     float diff = max(dot(Nnor, Ntolight), 0.f);
21
22     vec3 Nfromlight = normalize(lightDirection);
23     vec3 refDir = reflect(Nfromlight, Nnor);
24     vec3 camDir = normalize(camPos - FragPos);
25     float spec = pow(max(dot(camDir, refDir), 0.f), 128);
26
27     float i = amb + diff + spec;
28
29     return i;
30 }
```

Colour of a Fragment

- ◆ Finally, at the bottom of main() in the fragment shader, use the formula for calculating the colour of a fragment from the lecture notes.

```
32 void main()  
33 {  
34     //fragColour = vec4(col, 1.f);  
35     float phong = CalculateDirectionalIllumination();  
36     fragColour = vec4(phong * col * lightColour, 1.f);  
37 }
```

Reset the Light Direction



Positional Light

- ◆ Start by adding a new function to the fragment shader just above `main()`

```
float CalculatePositionalIllumination()
```

- ◆ Copy and paste the code from

```
CalculateDirectionalIllumination() to  
CalculatePositionalIllumination()
```

- ◆ Change the code in

`CalculatePositionalIllumination()` according to the lecture notes and the pseudocode below

Implementing Positional Light

```
33 float CalculatePositionalIllumination()  
34 {  
35     float amb = 0.1f;  
36  
37     vec3 Nnor = normalize(nor);  
38     vec3 NToLight = normalize(lightPos - FragPos);  
39     float diff = max(dot(Nnor, NToLight), 0.f);  
40  
41     vec3 NFromLight = -NToLight;  
42     vec3 refDir = reflect(NFromLight, Nnor);  
43     vec3 NcamDir = normalize(camPos - FragPos);  
44     float spec = pow(max(dot(NcamDir, refDir), 0.f), 128);  
45  
46     float d = length(lightPos - FragPos);  
47     float c = 1.5f;  
48     float l = .05f;  
49     float q = .02f;  
50     float att = 1.f / (c + (l*d) + (q*(d*d)));  
51  
52     float i = (amb + diff + spec) * att;  
53  
54     return i;  
55 }
```


In Lap9.cpp

- ◆ In the Fragment Shader, add a new uniform for light position

```
7    uniform vec3 lightColour;  
8    uniform vec3 lightDirection;  
9    uniform vec3 lightPos;  
10   uniform vec3 camPos;  
11
```

- ◆ Declare a variable in Lab9.cpp for storing the light position

```
16    //light direction variable here  
17    glm::vec3 lightDirection = glm::vec3(0.1f, -.81f, -.61f);  
18    glm::vec3 lightPos = glm::vec3(2.f, 6.f, 7.f);  
19    SCamera Camera;
```

More in Lap9.cpp

- ◆ Copy the light position to the shader in the render loop

```
237   glUniform3f(glGetUniformLocation(shaderProgram, "lightDirection"), lightDirection.x, lightDirection.y, lightDirection.z);  
238   glUniform3f(glGetUniformLocation(shaderProgram, "lightPos"), lightPos.x, lightPos.y, lightPos.z);  
239   glUniform3f(glGetUniformLocation(shaderProgram, "lightColour"), 0.f, 0.f, 1.f);  
240   glUniform3f(glGetUniformLocation(shaderProgram, "camPos"), Camera.Position.x, Camera.Position.y, Camera.Position.z);  
241
```

- ◆ And in the `processKeyboard()` function set the light position to the camera position when the space bar is pressed

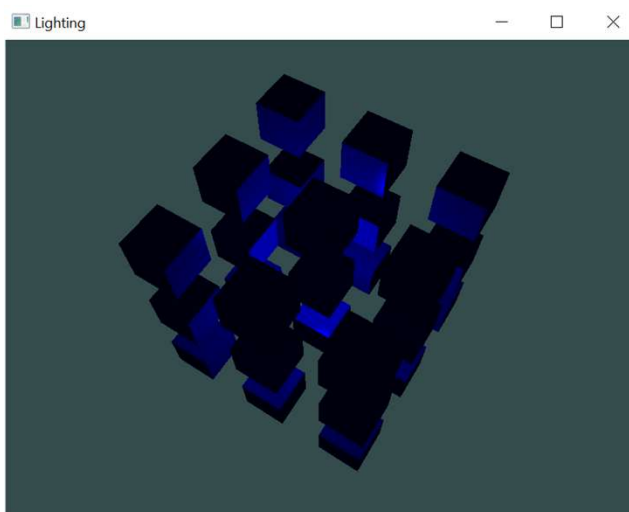
```
137   if (glfwGetKey(window, GLFW_KEY_SPACE) == GLFW_PRESS)  
138   {  
139       lightDirection = Camera.Front;  
140       lightPos = Camera.Position;  
141   }
```

Light Illumination

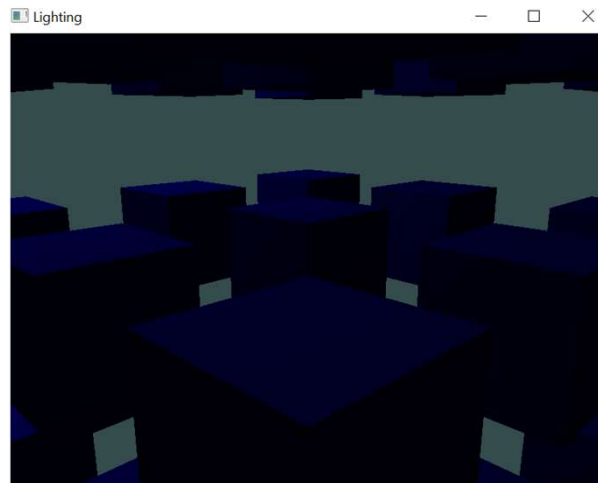
- ◆ Finally, call the new function in your fragment shader to calculate the positional light illumination

```
57 void main()
58 {
59     //fragColour = vec4(col, 1.f);
60     //float phong = CalculateDirectionalIllumination();
61     float phong = CalculatePositionalIllumination();
62     fragColour = vec4(phong * col * lightColour, 1.f);
63 }
```

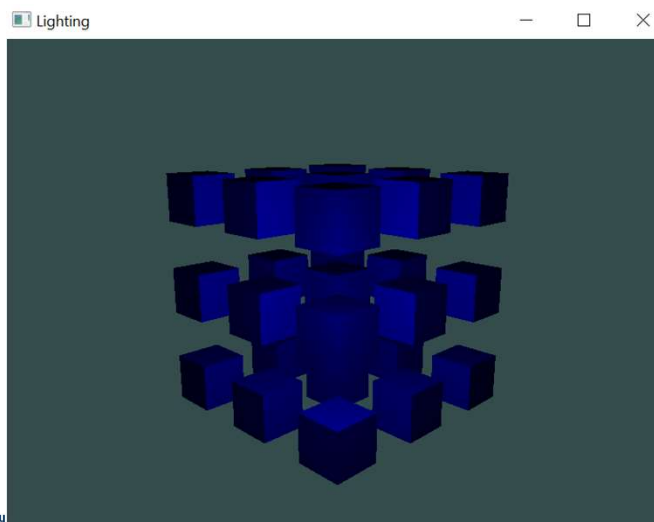
Results with Positional Light



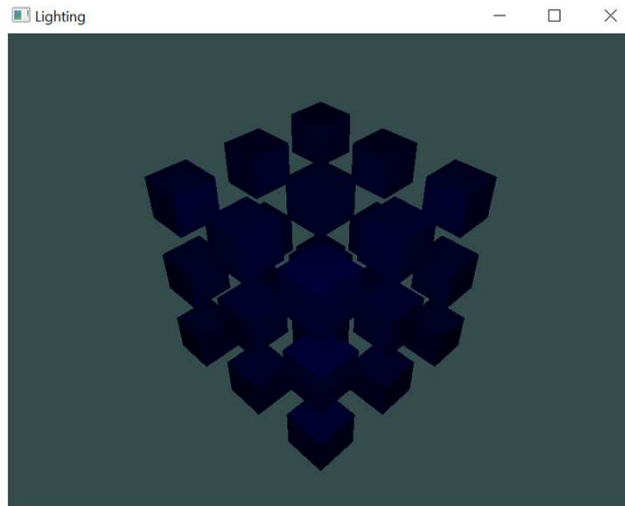
More Results



More Results



More Results



Spot Light

- ◆ Change the positional light to a spot light.
- ◆ Start by adding a new function to the fragment shader just above `main()`

```
float CalculateSpotIllumination()
```
- ◆ Copy and paste the code from `CalculatePositionalIllumination()` to `CalculateSpotIllumination()`
- ◆ Change the code in `CalculateSpotIllumination()` according to the lecture notes and the pseudocode below

Implementing Spot Light

```
float CalculateSpotIllumination()
{
    float amb = 0.1f;

    vec3 Nnor = normalize(nor);
    vec3 NToLight = normalize(lightPos - FragPos);
    float diff = max(dot(Nnor, NToLight), 0.f);

    vec3 NFromLight = -NToLight;
    vec3 refDir = reflect(NFromLight, Nnor);
    vec3 NcamDir = normalize(camPos - FragPos);
    float spec = pow(max(dot(NcamDir, refDir), 0.f), 128);

    float d = length(lightPos - FragPos);
    float c = 1.5f;
    float l = .05f;
    float q = .02f;
    float att = 1.f / (c + (l*d) + (q*(d*d)));

    float phi = cos(radians(15.f));
    vec3 NSpotDir = normalize(lightDirection);
    float theta = dot(NFromLight, NSpotDir);

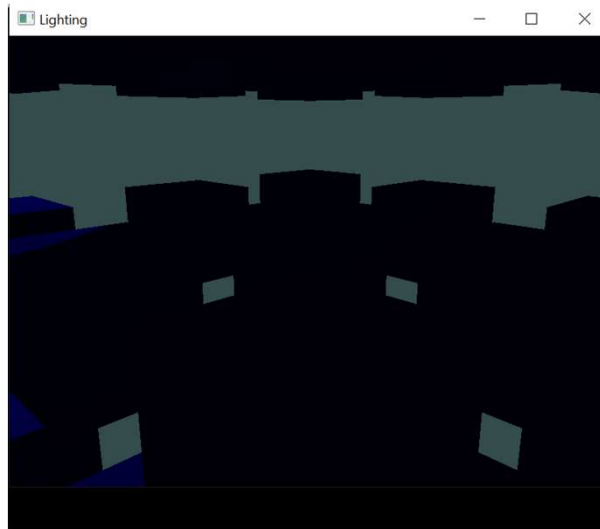
    float i;
    if(theta > phi)
    {
        i = (amb + diff + spec) * att;
    }
    else
    {
        i = (amb) * att;
    }

    return i;
}
```

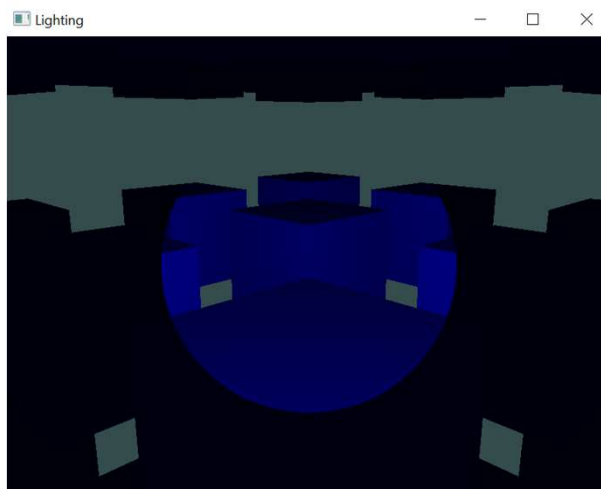
Other Changes

```
95 //fragColour = vec4(col, 1.f);
96 //float phong = CalculateDirectionalIllumination();
97 //float phong = CalculatePositionalIllumination();
98 float phong = CalculateSpotIllumination();
99 fragColour = vec4(phong * col * lightColour, 1.f);
```

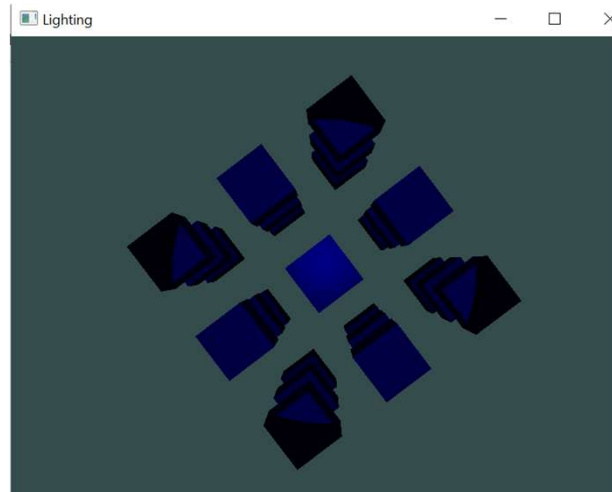
Results with Spot Light



Results with Changes of View



Results with Changes of View



References

- ◆ *COMP3069 Lab9 notes*
- ◆ *Interactive Computer Graphics: A Top-Down Approach with Shader-Based OpenGL, Chapter 5*